

Step 2 — Wrap It as a Single MCP Tool — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# mcp_server.py
from mcp.server.mcpserver import MCPServer
from pipeline import run_pipeline

mcp = MCPServer("deep-research-service")

@mcp.tool()
def deep_research(topic):
    result = run_pipeline(topic, max_rounds=3)
    return result.final_summary

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

This works. It's missing type hints, a real docstring, config, and logging — all fine for a first pass proving the wrapper itself calls the pipeline correctly.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with a real description, config, and logging

```
# mcp_server.py
from mcp.server.mcpserver import MCPServer

from pipeline import run_pipeline
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

mcp = MCPServer("deep-research-service")

@mcp.tool()
def deep_research(topic: str) -> str:
    """Research a topic and return a fact-checked summary. Internally this
    runs a Researcher/Fact-Checker loop with a bounded number of revision
    rounds and can take several seconds -- the caller only sees the final,
    approved (or best-effort) summary."""
    logger.info("deep_research called for topic=%r", topic)
    result = run_pipeline(topic, max_rounds=config.max_revision_rounds)
    logger.info(
        "deep_research finished for topic=%r: approved=%s, rounds=%d",
        topic, result.approved, result.revision_count,
    )
    return result.final_summary

if __name__ == "__main__":
    logger.info("Starting deep-research-service over stdio")
    mcp.run(transport="stdio")
```

Expected behavior: `mcp dev mcp_server.py` opens the Inspector; `deep_research` appears as the only tool. Calling it with `{"topic": "Retrieval-Augmented Generation (RAG)"}` takes a few seconds (multiple real LLM calls happening inside `run_pipeline`), then returns the final, fact-checked summary as a plain string.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a minimal client script, to prove it end to end without the Inspector

```
# client_step2_check.py -- a throwaway script, not part of the final project
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(command="python", args=["mcp_server.py"]
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            print("Tools this server offers:", [t.name for t in tools.tools])

            result = await session.call_tool(
                "deep_research",
                {"topic": "Retrieval-Augmented Generation (RAG)"},
            )
            print("Result:", result)

asyncio.run(main())
```

Expected output:

```
Tools this server offers: ['deep_research']
Result: <CallToolResult with the final fact-checked summary inside it>
```

This is the same `stdio_client` / `ClientSession` shape Project 7's Step 4 built — proof this server works with any real MCP client, not just the Inspector's UI.

Approach 2 — hardened against a stray `print()` anywhere in the import chain

```

# mcp_server.py
from mcp.server.mcpserver import MCPServer

from pipeline import run_pipeline
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

# NEVER call print() anywhere in this file, or in researcher.py, fact_checker.py,
# or pipeline.py -- on the stdio transport, stdout IS the protocol channel. A
# stray print() in ANY of those files corrupts every message the client reads
# after it, not just ones written directly in this file.

mcp = MCPServer("deep-research-service")

@mcp.tool()
def deep_research(topic: str) -> str:
    """Research a topic through a Researcher/Fact-Checker loop and return a
    fact-checked summary. This can take several seconds -- it may make
    several real LLM calls internally before answering."""
    logger.info("deep_research called for topic=%r", topic)
    result = run_pipeline(topic, max_rounds=config.max_revision_rounds)
    logger.info(
        "deep_research finished for topic=%r: approved=%s, rounds=%d",
        topic, result.approved, result.revision_count,
    )
    return result.final_summary

if __name__ == "__main__":
    logger.info("Starting deep-research-service over stdio")
    mcp.run(transport="stdio")

```

Expected behavior: identical to Intermediate from the client's point of view. The difference only shows up the day someone (or future-you) adds a debug `print()` to `researcher.py` while chasing a prompt-tuning issue — this comment, sitting in the one file every future change touches, is what stops that from silently breaking the whole server.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the wrapper works, tested by hand through the Inspector. Approach 1 proves the exact same thing with real client code — the mechanism any actual MCP client (Claude Desktop, another agent) would use. Approach 2 changes no behavior; it documents, right where it matters most, the one

constraint (no `print()`, anywhere in the import chain) that's cheap to write down and expensive to debug once violated by accident.

Which one should you actually write? Intermediate Approach 1 is what belongs in `mcp_server.py` for the rest of this project. Run Advanced Approach 1's client script once, by hand — confirming `deep_research` works through a real `ClientSession`, not just the Inspector, is worth the two minutes it takes. Keep Approach 2's comment; it costs nothing and saves a genuinely confusing debugging session later.